# Secure Station Dispatch

ArUco-Driven AMR Station Identification,
HMAC Security & Gazebo Navigation

**Amin Ahmed G**

B.E. Robotics & Automation Engineering

Anna University

ROS 2 Jazzy | Gazebo Sim (Harmonic) | OpenCV 4.7+

Package: secure_station_dispatch

# Table of Contents

1. Title & Objective
2. Tech Stack
3. System Architecture
4. Phase 1: Dispatch Pipeline
5. Phase 2: HMAC Security & Attack Demonstration
6. Phase 3: Gazebo Simulation & Autonomous Navigation
7. Testing & Results
8. Challenges Faced
9. Conclusion
10. GitHub Repository & Package Structure

# 1. Title & Objective

## Project Title

ArUco Station Identification & Task Triggering for Autonomous Mobile Robots -- with HMAC Security and Gazebo Simulation

## Objective

The goal of this project is to build a complete AMR dispatch system using ROS 2. An autonomous mobile robot tours a walled industrial facility, reads ArUco fiducial markers at four stations (Inspection, Assembly, Storage, Charging), authenticates each detection using HMAC-SHA256 signatures, and executes station-specific tasks through ROS 2 action interfaces. The robot navigates autonomously in Gazebo Sim using obstacle-aware path planning with stall detection and recovery.

The project is structured in three additive phases:

**Phase 1 -- Dispatch Pipeline:** ArUco detection, state machine, action-based task execution with JSONL event logging.

**Phase 2 -- Security Hardening:** HMAC-SHA256 message authentication and pose-consistency verification, with two working attack demonstrations.

**Phase 3 -- Gazebo Simulation:** A generated walled facility, differential-drive AMR with a sensor mast, obstacle-aware scripted tour with stall watchdog and recovery.

## 2. Tech Stack

| Component | Technology |
|---|---|
| **ROS 2 Distribution** | ROS 2 Jazzy Jalisco (Python 3.12) |
| **Simulator** | Gazebo Sim (Harmonic), Ogre2 renderer |
| **Computer Vision** | OpenCV cv2.aruco (DICT_4X4_50), cv_bridge |
| **Communication** | ROS 2 Topics, ROS 2 Actions (Client/Server) |
| **Custom Interfaces** | StationMarker.msg, StationTask.action |
| **Security** | HMAC-SHA256 (hashlib), per-station pre-shared keys |
| **Navigation** | Custom A* path planner, P-controller, stall watchdog |
| **Configuration** | YAML (station_registry.yaml) -- single source of truth |
| **Logging** | JSONL (dispatch_events.jsonl, tour_report.json) |
| **Build System** | colcon + CMake (ament_cmake) + Python setup.py |

# 3. System Architecture

I split the system into separate ROS 2 nodes so I could develop, test, and debug each part -- perception, decision-making, task execution, security, and navigation -- independently. This mirrors how real AMR systems separate concerns into modular packages.

## Core Nodes

**`marker_detector_node.py`:** Subscribes to /camera/image_raw, detects ArUco markers using OpenCV DICT_4X4_50, computes confidence from marker perimeter, signs each detection with HMAC-SHA256, and publishes a StationMarker message.

**`dispatcher_node.py`:** Runs an IDLE / TASK_IN_PROGRESS state machine. Verifies HMAC signatures, checks pose consistency against station_registry.yaml, applies per-marker re-dispatch cooldowns, and sends action goals to the task server. Logs every event to dispatch_events.jsonl.

**`task_simulator_node.py`:** Serves as the ROS 2 action server for /station_task. Receives goals, runs a 5-step simulated work sequence with progress feedback, and returns a success result with execution time.

**`waypoint_tour_node.py`:** Obstacle-aware scripted tour. Uses a visibility-graph A* planner, turn-then-arc differential-drive controller, and a stall watchdog with reverse-and-replan recovery. Reads all geometry from station_registry.yaml.

## ROS 2 Topics & Interfaces

| Topic | Type | Direction | Purpose |
|---|---|---|---|
| **/camera/image_raw** | sensor_msgs/Image | Gazebo -> detector | Mast camera feed |
| **/station_marker** | StationMarker | detector -> dispatcher | Detection + pose + HMAC |
| **/station_task** | StationTask (Action) | dispatcher -> simulator | Goal / feedback / result |
| **/dispatch_status** | String (JSON) | dispatcher -> tour | Every logged outcome |
| **/cmd_vel** | geometry_msgs/Twist | tour -> Gazebo | Velocity command |
| **/odom** | nav_msgs/Odometry | Gazebo -> tour | Pose feedback |
| **/mock_pose** | PoseStamped | tour -> detector | Pose stamped into detections |

## Single Source of Truth

station_registry.yaml is the only place station geometry is defined. The Gazebo world, the navigation planner, the detector, and the security checks all read from this single file. A drift-guard script (check_world_sync.py) fails the build if any consumer disagrees with the registry.

# 4. Phase 1: Dispatch Pipeline

## Custom Interfaces

StationMarker.msg carries the detection data from the detector to the dispatcher:

```
int32      marker_id
string     station_name
float64[7] robot_pose
float64    confidence
builtin_interfaces/Time stamp
string     hmac_signature    # Added in Phase 2
```

StationTask.action handles multi-step task execution:

```
# Goal
string task_type
string station_name
---
# Result
bool    success
string  summary
float64 execution_time
---
# Feedback
string  status
float64 progress
```

## Station Registry

| Marker ID | Station | Task Type | Berth Pose | Duration |
|---|---|---|---|---|
| **0** | Inspection Station | Inspection | [1.0, 2.0, 0.0] | ~3.0 s |
| **1** | Assembly Station | Assembly | [4.5, 1.0, 0.0] | ~3.5 s |
| **2** | Storage Station | Storage | [4.5, 5.0, 0.0] | ~2.7 s |
| **3** | Charging Station | Charging | [0.5, 5.5, 0.0] | ~3.5 s |

## State Machine

The dispatcher starts in IDLE. When a valid marker message arrives, it transitions to TASK_IN_PROGRESS and ignores all new detections to prevent interrupting the current task. When the action server returns the result, the dispatcher logs the outcome to dispatch_events.jsonl and drops back to IDLE.

# 5. Phase 2: HMAC Security & Attack Demonstration

## Threat Model

An unsecured dispatch pipeline is vulnerable: a forged ROS message or a printed marker could drive a facility robot anywhere. Phase 2 adds two security layers to authenticate what the camera claims to see.

## Security Implementation

**1. HMAC-SHA256 Message Authentication:** Every StationMarker detection is signed by the detector using HMAC-SHA256. The payload is marker_id + station_name, keyed with a per-station pre-shared secret from station_registry.yaml. The dispatcher calls verify_hmac() and rejects any message with an invalid or missing signature, logging AUTH_FAIL.

**2. Pose Consistency Verification:** Even with a valid signature, a replayed or spoofed detection from the wrong physical location is rejected. check_pose() compares the reported robot_pose against the station expected_pose within pose_tolerance_m (0.5 m). If the robot is too far, the message is rejected with LOCATION_MISMATCH.

## Attack-Mitigation Matrix

| Attack Vector | Mitigation | Outcome Logged |
|---|---|---|
| **Forged StationMarker on topic** | HMAC-SHA256 signature verification | AUTH_FAIL |
| **Valid signature replayed from wrong location** | check_pose() vs expected_pose +/- tolerance | LOCATION_MISMATCH |
| **Genuine board read from too far away** | Confidence gate doubles as range gate | Never published |
| **Same bay dispatched repeatedly** | Per-marker cooldown after completion | COOLDOWN |

## Attack Demonstrations

**attacks/spoof_marker_publish.py:** Publishes a forged StationMarker with an invalid HMAC signature directly onto the /station_marker topic. With security enabled, the dispatcher rejects the message and logs AUTH_FAIL.

**attacks/location_mismatch_attack.py:** Computes a valid HMAC signature (using the real key from the registry) but reports an impossible robot pose of [99.0, 99.0, 0.0]. The HMAC passes, but the pose check fails and the dispatcher logs LOCATION_MISMATCH.

## Security Module Code

```
# security/checks.py (condensed)

def verify_hmac(msg, registry):
    key = registry[msg.marker_id]["hmac_key"].encode()
    payload = f"{msg.marker_id}{msg.station_name}".encode()
    expected = hmac.new(key, payload, sha256).hexdigest()
    return hmac.compare_digest(expected, msg.hmac_signature)

def check_pose(msg, registry):
    expected = registry[msg.marker_id]["expected_pose"]
    tolerance = registry[msg.marker_id]["pose_tolerance_m"]
    dist = sqrt(sum((a - b)**2 for a, b in zip(robot_pos, expected)))
    return dist <= tolerance
```

# 6. Phase 3: Gazebo Simulation & Autonomous Navigation

## Facility & World Generation

The Gazebo world is not hand-crafted -- it is generated from station_registry.yaml by scripts/generate_world.py. This ensures the simulator, planner, and security checks can never disagree about station positions. The facility is an 8.5 m x 9.5 m walled enclosure with four perimeter bays, each carrying a three-part sign stack (nameplate, 1.10 m carrier board with a 0.70 m ArUco marker, and an info plate).

## AMR Robot Model

A differential-drive AMR with a 0.40 m x 0.30 m chassis, two powered wheels, two passive casters, and a 0.85 m sensor mast with a front-facing camera. The camera sits at the same height as the marker board centres, ensuring clean framing at the 1.65 m berth standoff distance.

## Navigation Architecture

The navigation module (secure_station_dispatch/navigation/) is deliberately ROS-free so the entire tour can be replayed offline in the test suite without Gazebo or ROS.

**`planner.py:`** Visibility-graph A* path planner with two clearance layers. Hard layer = robot radius (0.28 m). Soft layer = hard + safety margin (0.43 m total). Berths may sit inside a soft halo but never a hard footprint.

**`controller.py:`** Turn-then-arc differential-drive controller. Heading errors beyond 0.55 rad are turned on the spot. Linear speed is proportional to range and attenuated by heading error, decelerating into each berth.

**`StallWatchdog:`** Detects when the robot is commanded to move but does not (< 4 cm in 3 s). Triggers reverse 0.35 m + turn 0.70 rad recovery, then re-plans from the freed pose, up to 3 times.

**`perception_budget.py:`** Pinhole camera model computing marker legibility. Confidence = min(perimeter_px / 1200, 1.0). Makes a board legible only from its own berth, preventing false dispatches while driving past.

## Perception Budget

| Camera Range | Marker px | Confidence | Verdict |
|---|---|---|---|
| **< 1.33 m** | -- | -- | Board clips frame, detection drops |

| | | | |
|---|---|---|---|
| **1.50 m (berth)** | 259 | 0.86 | Accept -- this is the berth |
| **1.72 m** | 226 | 0.75 | Accept (gate threshold) |
| **1.90 m** | 204 | 0.68 | Reject |
| **2.50 m** | 155 | 0.52 | Reject |

# 7. Testing & Results

## Test Suites

| Suite | Command | Needs | Time |
|---|---|---|---|
| **Navigation + perception regressions** | bash test/run_navigation_checks.sh | Nothing | ~1 s |
| **Phase 1 pipeline integration** | bash test/run_test.sh | ROS 2 | ~60 s |
| **Phase 2 security integration** | bash test/run_security_test.sh | ROS 2 | ~30 s |
| **Headless dispatch integration** | bash test/run_gazebo_tour_test.sh | ROS 2 | ~90 s |
| **Full Gazebo demo** | ros2 launch ... gazebo_demo.launch.py | ROS 2 + Gazebo | ~2 min |

## Phase 1 Pipeline Test Results

```
============================================================
  PIPELINE TEST SUMMARY
============================================================
  Events logged    : 8
  DISPATCHED events: 4
  COMPLETED events : 4

  Marker 0: dispatched=True  completed=True   PASS
  Marker 1: dispatched=True  completed=True   PASS
  Marker 2: dispatched=True  completed=True   PASS
  Marker 3: dispatched=True  completed=True   PASS

  OVERALL: PASS -- Phase 1 pipeline working
============================================================
```

## Phase 2 Security Test Results

```
============================================================
  PHASE 2 SECURITY TEST SUMMARY
============================================================
  Total security events logged: 5
  Valid dispatches           : 1
  Valid completions          : 1
  AUTH_FAIL rejections       : 1
  LOCATION_MISMATCH rejections: 1
```

```
  Test 1 (Valid signed marker):        PASS
  Test 2 (Spoofed HMAC rejected):      PASS
  Test 3 (Location mismatch rejected): PASS

  OVERALL: PASS -- All Phase 2 security protections working
============================================================
```

## Phase 3 Navigation Test Results

```
$ bash test/run_navigation_checks.sh
  world / registry synchronisation ...... 12 collision models inside footprints
  planner + perception regression ....... 17/17 passed
  offline tour simulation ............... 6/6 passed

  tour duration      73.5 s simulated
  worst berth error  9.9 cm  (tolerance 10 cm)
  obstacle contacts  0
  minimum clearance  0.528 m
```

## Full Tour Station Arrival Log

| Station | Target Coords | Arrived Pose | Error | Status |
|---|---|---|---|---|
| **Inspection** | [1.00, 2.00] | [0.95, 2.21] | 0.219 m | DISPATCHED -> COMPLETED |
| **Assembly** | [4.50, 1.00] | [4.31, 1.10] | 0.217 m | DISPATCHED -> COMPLETED |
| **Storage** | [4.50, 5.00] | [4.49, 4.78] | 0.217 m | DISPATCHED -> COMPLETED |
| **Charging** | [0.50, 5.50] | [0.72, 5.48] | 0.219 m | DISPATCHED -> COMPLETED |

# 8. Challenges Faced

## 1. ROS 2 Jazzy CMake Target Collision

The main build issue I ran into while setting up the package was a CMake target name collision during colcon build. In standard ROS 2 Python packages, CMakeLists.txt uses ament_python_install_package(). But because I also defined custom message and action interfaces using rosidl_generate_interfaces() in the same package, ROS 2 Jazzy generated internal CMake targets that conflicted. The build kept failing with target redefinition errors.

To fix it, I removed ament_python_install_package() and used install(DIRECTORY) to place the Python files directly into site-packages. The interpreter version is queried at configure time rather than hard-coded, so the same CMakeLists.txt builds on both Humble (3.10) and Jazzy (3.12).

## 2. Open-Loop Navigation Producing False Log Successes

The waypoint_tour_node was originally open-loop -- it published a target coordinate to /mock_pose and commanded the robot to drive forward for a fixed time. Because the dispatcher only checked /mock_pose, the dispatch log showed 4/4 COMPLETED events, making it look like the tour passed. But watching Gazebo showed the robot just driving straight near the origin, never reaching any station.

| Station | Target | Open-Loop Pose | OL Error | Closed-Loop Pose | CL Error |
|---|---|---|---|---|---|
| **Inspection** | [1.00, 2.00] | [0.50, 0.00] | 2.06 m | [1.09, 1.89] | 0.147 m |
| **Assembly** | [4.50, 1.00] | [1.00, 0.00] | 3.64 m | [4.36, 1.05] | 0.148 m |
| **Storage** | [4.50, 5.00] | [1.50, 0.00] | 5.83 m | [4.50, 4.85] | 0.148 m |
| **Charging** | [0.50, 5.50] | [2.00, 0.00] | 5.70 m | [0.65, 5.49] | 0.149 m |

I rewrote waypoint_tour_node to use /odom for real position feedback and built a closed-loop proportional controller.

## 3. Robot Collision with Marker Panel Structures

During Phase 3, the robot drove into the Assembly Station marker panel while departing for Storage. It wedged against the panel, kept commanding forward velocity for the full 20 s leg timeout, continued the tour from a corrupted pose, and still reported success -- while two stations were never dispatched.

This was four defects that lined up:

**Layout defect:** Marker panels placed 0.40 m from the berth with solid collision boxes directly in the driving path.

**Planning defect:** The tour drove straight lines between berths. Nothing in the node knew obstacles existed.

**Control defect:** No stall detection. The timeout exit was indistinguishable from a successful arrival.

**Reporting defect:** The tour printed 'All stations visited successfully!' while 3.6 m from its target.

The fix was comprehensive: I made station_registry.yaml the single source of truth, rebuilt the facility layout with bays on the perimeter, wrote an A* visibility-graph planner with obstacle inflation, added a stall watchdog with reverse-and-replan recovery, and made arrival/timeout distinct outcomes. Two regression tests pin this specific failure so it can never come back.

## 4. Station Signage Collision Geometry

After adding decorative station signage (posts, backing panels, label shelves, foot plates), the robot chassis front edge (extending 0.20 m ahead of center) would physically contact the sign base plates before the stopping controller considered itself arrived. The fix was to simplify the world back to minimal marker panels with ultra-thin collision geometry (0.01 m depth).

# 9. Conclusion

The secure_station_dispatch package delivers a complete, tested AMR dispatch system:

- Phase 1 established a working perception to dispatch to execution pipeline with custom ROS 2 interfaces, a finite state machine, and structured JSONL event logging.
- Phase 2 added real security -- HMAC-SHA256 authentication and pose-consistency checking reject forged and replayed messages. Two working attack scripts demonstrate both rejection pathways.
- Phase 3 made the robot physically navigate a generated Gazebo facility using obstacle-aware A* planning, closed-loop differential-drive control, and a stall watchdog that detects collisions and recovers automatically.
- The single-source-of-truth architecture (station_registry.yaml) means the simulator, planner, detector, and security checks can never disagree about where a station is.
- Five test suites -- from a 1-second ROS-free regression to a full 2-minute Gazebo demo -- verify every layer independently.

# 10. GitHub Repository & Package Structure

GitHub: https://github.com/Amin-Ahmed-G/secure_station_dispatch

```
secure_station_dispatch/
|-- config/station_registry.yaml          <- single source of truth
|-- msg/StationMarker.msg
|-- action/StationTask.action
|-- launch/
|   |-- gazebo_demo.launch.py             full simulated demo
|   |-- station_dispatch.launch.py        nodes only
|-- secure_station_dispatch/
|   |-- marker_detector_node.py           camera -> ArUco -> StationMarker
|   |-- dispatcher_node.py                state machine, security, cooldown
|   |-- task_simulator_node.py            StationTask action server
|   |-- waypoint_tour_node.py             obstacle-aware scripted tour
|   |-- navigation/                       ROS-free: planner, controller, budget
|   |-- security/checks.py                HMAC + pose consistency
|-- worlds/industrial_station_world.sdf    <- generated from registry
|-- models/materials/textures/             <- generated signage
|-- scripts/
|   |-- generate_world.py                 registry -> SDF
|   |-- generate_marker_boards.py         registry -> signage + verification
|   |-- check_world_sync.py               drift guard
|   |-- render_facility_map.py            docs figures
|-- test/
|   |-- test_waypoint_planner.py          planner + perception regressions
|   |-- test_tour_simulation.py           offline full-tour simulation
|   |-- test_dispatch_pipeline.py         Phase 1 integration
|   |-- test_security_checks.py           Phase 2 integration
|   |-- run_*.sh                          suite runners
|-- attacks/                              spoofing + location-mismatch demos
|-- docs/                                 architecture, root cause, runbook
```